Project Report on

“n9n: AI WORKFLOW AUTOMATION USING AGENTIC ORCHESTRATION AND SEO”

Submitted in partial fulfilment of the requirement for award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

By

Anmol Bhardwaj – 2300681530017
Avi Gupta – 2300681530032
Jatin Raghav – 2300681530054
Keshav Garg – 2300681530060

Under the Guidance of

Mr. Ankit Aggarwal
Ms. Mohini Singh (Department Supervisor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



MEERUT INSTITUTE OF ENGINEERING AND TECHNOLOGY, MEERUT

AFFILIATED TO



DR. A. P. J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

DEC 2026

Table of Content

Description		Page No.
Declaration		2
Certificate		3
Acknowledgement		4
Abstract		5
List of Figures		7
List of Tables		8
List of Abbreviations		9
Chapter 1	**Introduction**	1
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Aim	2
	1.4 Objectives	2
	1.5 Scope of the Project	2
	1.6 Organisation of the Report	2
Chapter 2	**Literature Review**	4
	2.1 Traditional Workflow Automation	4
	2.2 Single-Agent Reasoning and Tool Use	4
	2.3 Multi-Agent and Orchestration Frameworks	6
	2.4 Evaluation of Agentic Systems	7
	2.5 SEO and Generative Engine Optimisation	8
	2.6 Surveys of LLM Agents	8
	2.7 Comparative Analysis	8
	2.8 Summary of Reviewed Literature	10
	2.9 Research Gap	10
Chapter 3	**Methodology**	12
	3.1 Development Approach	12
	3.2 Requirements	12
	3.3 System Architecture	13
	3.4 Workflow Model	14
	3.5 Execution Engine	15
	3.6 Database Design	16
	3.7 API Design	17
	3.8 Implementation of the Components	18
	3.9 Deployment	19

3.10 Design of the Agentic Layer 20
3.11 Design of the SEO Layer 21
3.12 Testing Strategy 22
3.13 Tools and Technologies 22
Chapter 4 Results and Discussion 24
4.1 Implementation Status 24
4.2 User Interface 24
4.3 Experimental Setup 28
4.4 End-to-End Latency 28
4.5 Throughput and Worker Concurrency 29
4.6 Replay from a Chosen Step 30
4.7 Technical SEO Audit Workflow 30
4.8 Fault Injection 31
4.9 Deployment Verification 32
4.10 Discussion 33
4.11 Limitations 33
Chapter 5 Conclusion and Future Scope 34
5.1 Conclusion 34
5.2 Future Scope 34
References 36
Appendix A – Running the Project 38
Appendix B – Papers Produced 39

DECLARATION

We hereby declare that the project entitled “n9n: AI Workflow Automation using Agentic Orchestration and SEO”, which is being submitted as the Project in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) to Meerut Institute of Engineering and Technology, Meerut (U.P.), is an authentic record of our genuine work done under the guidance of Mr. Ankit Aggarwal, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Meerut Institute of Engineering and Technology, Meerut.

The matter embodied in this report has not been submitted by us for the award of any other degree or diploma. The research papers listed in Appendix B were written by the same team as part of this project.

Date:

Place: MIET, Meerut

Name of Students

Anmol Bhardwaj (2300681530017)

Avi Gupta (2300681530032)

Jatin Raghav (2300681530054)

Keshav Garg (2300681530060)

CERTIFICATE

This is to certify that the project report entitled “n9n: AI Workflow Automation using Agentic Orchestration and SEO” submitted by “**Anmol Bhardwaj (2300681530017), Avi Gupta (2300681530032), Jatin Raghav (2300681530054), Keshav Garg (2300681530060)**” has been carried out under the guidance of **“Mr. Ankit Aggarwal”**, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Meerut Institute of Engineering and Technology, Meerut. This project report is approved for Project (KCS 752) in the 7th semester in “Artificial Intelligence and Machine Learning” from Meerut Institute of Engineering and Technology, Meerut.

Supervisor Signature
Mr. Ankit Aggarwal

Department Supervisor Signature
Ms. Mohini Singh

Date:

ACKNOWLEDGEMENT

We express our sincere indebtedness towards our guide Mr. Ankit Aggarwal, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Meerut Institute of Engineering and Technology, Meerut, for his valuable suggestions, guidance and supervision throughout the work. Without his patronage and guidance the project would not have taken shape. We would also like to express our gratitude for his approval of the project and for his counselling and advice from time to time.

We are thankful to our department supervisor Ms. Mohini Singh for her support, feedback and encouragement during the review stages of this project.

We would also like to thank our Head of Department, Dr. Rambir Singh, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Meerut Institute of Engineering and Technology, Meerut, for his expert advice and counselling from time to time.

We owe sincere thanks to all the faculty members of the department for their kind guidance and encouragement, and to the authors and maintainers of the open-source projects on which n9n is built.

Date:

Name of Students

Anmol Bhardwaj (2300681530017)

Avi Gupta (2300681530032)

Jatin Raghav (2300681530054)

Keshav Garg (2300681530060)

ABSTRACT

Rule-based workflow platforms such as Zapier, Make, n8n and Node-RED execute fixed trigger-to-action chains reliably, but they cannot interpret context or recover from states they were not programmed for. Large language model (LLM) agents can reason, plan and call tools, but benchmark studies show that they remain unreliable on open-ended tasks and that their execution is neither reproducible nor auditable. This project studies how the two paradigms can be combined into one production-oriented automation platform, with search engine optimisation (SEO) auditing as the application domain.

We reviewed twenty-four works (twenty-two research papers and the documentation of two platforms) on reasoning-and-acting loops, tool use, multi-agent coordination, agent evaluation and generative engine optimisation, compared them on the properties a production workflow engine needs, and identified the gap: no reviewed system provides durable, replayable, side-effect-safe execution as the base on which agentic decisions are made.

We then designed and built **n9n**, a visual workflow automation platform in three layers. The deterministic substrate is implemented: a Go API that validates workflows as acyclic single-trigger graphs and stores immutable versions, a Rust worker that executes runs under leases with per-step and per-attempt records, PostgreSQL 16 for all state, a Flutter web editor and a React Native mobile app. It runs locally as one Docker Compose stack and in production on Cloudflare Workers, where a TypeScript port of the API is kept in sync with the Go version by automated parity tests. The agentic layer (a prompt-to-workflow builder with a validate-and-repair loop, and a run-failure explainer whose retry verdict is enforced in code) and the SEO layer are specified on top of it.

On a local deployment, a seven-node branching workflow completed in a median of 260 ms end to end with 7 ms of recorded step time; a burst of 100 queued runs was drained at 71.3, 178.5 and 158.6 runs per second at worker concurrency 1, 4 and 8; replaying a nine-node run from its sixth node re-executed only 4 steps and reused 5; a technical SEO audit workflow built from existing nodes classified status, redirect, retry and missing-title cases correctly; and a worker killed during a side-effecting step was recovered as queued and completed when the step was idempotent, and marked uncertain when it was not. These results characterise the substrate only; the agentic and SEO reasoning components are not yet measured, and an evaluation plan for them is given.

Keywords agentic AI, workflow orchestration, large language models, durable execution, replay, SEO automation, generative engine optimisation, Go, Rust, PostgreSQL

LIST OF FIGURES

Fig. 3.1 Three-layer architecture of n9n 13

Fig. 3.2 Containers, boundaries and data flows of the deterministic substrate (from the project's architecture diagram; password hashing has since changed from bcrypt to PBKDF2-SHA256, Section 3.8) 14

Fig. 3.3 Workflow and run lifecycle, from building a draft to a terminal run status 15

Fig. 3.4 Database schema as first designed (later migrations add step_attempts and the replay columns described above) 17

Fig. 3.5 Internal structure of the web, mobile, API and worker components 18

Fig. 3.6 Technical SEO audit workflow built from existing n9n nodes 22

Fig. 4.1 Sign-in screen of the web editor 25

Fig. 4.2 Workflow list with published and draft workflows and recent runs 25

Fig. 4.3 Canvas editor showing the seven-node branching workflow, the node library and recent runs 26

Fig. 4.4 Execution inspector for a run of the branching workflow: timeline, run canvas and per-node status, with the untaken branches marked skipped 26

Fig. 4.5 The technical SEO audit workflow in the editor 27

Fig. 4.6 Inspection of an SEO audit run of https://example.com/: the HTTP fetch succeeded and the page passed the title check 27

Fig. 4.7 Seven-node branching workflow used for the latency and throughput experiments 28

Fig. 4.8 Throughput by worker concurrency for a burst of 100 queued runs 29

Fig. 4.9 Steps executed and reused when replaying the nine-node chain from its sixth node 30

LIST OF TABLES

Table 2.1 Rule-based automation versus agentic orchestration 8
Table 2.2 Comparison of reviewed works on substrate properties 9
Table 2.3 Year-wise summary of key reviewed works 10
Table 3.1 Functional requirements 12
Table 3.2 Non-functional requirements 13
Table 3.3 Node catalogue (version 2) 14
Table 3.4 Database tables 16
Table 3.5 Main public API routes 17
Table 3.6 Mapping of n9n components to the literature 21
Table 3.7 Tools and technologies used 22
Table 4.1 Implementation inventory 24
Table 4.2 End-to-end latency of the seven-node workflow (50 sequential runs) 28
Table 4.3 Burst of 100 queued runs drained by one worker 29
Table 4.4 Replay of a nine-node chain from its sixth node (10 repetitions) 30
Table 4.5 Technical SEO audit, one run per target 31
Table 4.6 Recovery after the worker is killed mid-action (60 s lease) 32
Table 4.7 Deployment verification 32
Table 5.1 Evaluation plan for the agentic and SEO layers 34

LIST OF ABBREVIATIONS

AES-GCM	Advanced Encryption Standard in Galois/Counter Mode
API	Application Programming Interface
CI	Continuous Integration
DAG	Directed Acyclic Graph
GEO	Generative Engine Optimisation
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSONB	Binary JSON column type in PostgreSQL
LLM	Large Language Model
OAuth	Open Authorization
PBKDF2	Password-Based Key Derivation Function 2
SEO	Search Engine Optimisation
SQL	Structured Query Language
UI	User Interface

CHAPTER 1 – INTRODUCTION

Workflow automation has historically relied on rigid, event-triggered rule chains: a trigger fires, a condition is checked and a predefined action executes. This model, popularised by Zapier, Make, n8n and Node-RED, suits repetitive, well-specified integration tasks, but it breaks down when a workflow must interpret ambiguous content, weigh alternatives or recover from an unexpected state. Large language models that can reason, plan and invoke external tools offer an alternative: *agentic* systems in which a model analyses a situation, decides on an action and executes it, possibly together with other agents [1], [9].

1.1 BACKGROUND

The two paradigms have complementary weaknesses. Rule engines are predictable and auditable but cannot decide. Agents can decide, but realistic benchmarks such as WebArena, AgentBench and GAIA show that they succeed on a minority of tasks, are sensitive to prompt and model changes, and leave no execution record that a person can replay [16], [17], [18]. A production system that uses an agent for a judgement call therefore needs a base layer that makes the deterministic parts of a workflow durable, replayable and safe against duplicated side effects, so that the agent's unreliability is contained rather than amplified.

Search engine optimisation (SEO) is a good test domain for this combination. Technical SEO checks (reachability, status codes, redirects, robots and sitemap files, presence of a title and metadata) have exact answers and should never be delegated to a model. Content assessments (relevance to a query, intent match, readability) are judgement calls that a model can make. Recent work on generative engine optimisation (GEO) [21] shows that visibility inside LLM-driven search is itself becoming an optimisation target, which makes a platform that combines fixed checks with model judgement timely.

1.2 PROBLEM STATEMENT

Existing automation platforms either execute fixed rules without judgement, or let an agent own a whole task with its state held in process memory. Neither records execution in a way that lets a failed or interrupted run be resumed, replayed from a chosen step, or checked for side effects that may have happened twice. The problem addressed by this project is to build a workflow platform whose execution is durable, replayable and honest about side effects, so that agentic and SEO decisions can later be added as ordinary, recorded steps.

1.3 AIM

To review the state of the art in agentic AI orchestration and workflow automation, and to design, implement and evaluate a scalable, production-oriented automation platform (n9n) whose deterministic substrate can host agentic decisions and technical and content-based SEO analysis.

1.4 OBJECTIVES

1. Survey the reasoning-and-acting, tool-use and multi-agent frameworks that underlie modern LLM agents, and the benchmarks used to evaluate them.
2. Compare rule-based workflow automation with agentic orchestration on control logic, adaptability, durability and evaluation, and state the research gap.
3. Design a three-layer platform: a deterministic substrate, an agentic layer and an SEO layer.
4. Implement the substrate: a Go API, a Rust worker, PostgreSQL storage, a Flutter web editor and a React Native mobile app, with automated tests.
5. Measure the substrate on latency, throughput, replay, fault recovery and a technical SEO audit workflow.
6. Deploy the platform locally with Docker Compose and publicly on Cloudflare Workers, and specify an evaluation plan for the agentic and SEO layers.

1.5 SCOPE OF THE PROJECT

The project covers the full deterministic substrate and its two clients, the local and cloud deployments, a technical SEO audit expressed with the existing nodes, and the design of the agentic layer. It does not include calls to a language model: the prompt-to-workflow builder, the failure explainer and the agent node are specified and switched off by a feature flag (`GET /api/features` reports `ai: false` until a model key is configured). The content side of the SEO layer is likewise specified but not measured.

1.6 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature, compares the reviewed works and states the research gap. Chapter 3 describes the methodology: requirements, architecture, workflow model, execution engine, database and API design, implementation, deployment and testing. Chapter 4 presents the implemented system and the measured results, and discusses them together with

the limitations. Chapter 5 concludes and gives the future scope, including the evaluation plan for the agentic and SEO layers. The references and two appendices (running the project, and the papers produced) follow.

CHAPTER 2 – LITERATURE REVIEW

The review covers twenty-four works: twenty-two research papers and the public documentation of n8n and LangGraph, selected for their relevance to running LLM decisions inside an automated workflow. Bibliographic details were checked against the arXiv record or the publisher page of each work. The works fall into four strands, listed in the order of their importance to n9n: single-agent reasoning and tool use; multi-agent and orchestration frameworks; evaluation; and SEO together with survey papers. For each work the review states its contribution, its limitation for production workflow use, and its relevance to n9n.

2.1 TRADITIONAL WORKFLOW AUTOMATION

Traditional platforms such as Zapier, Make (formerly Integromat), n8n and Node-RED connect services through visual, node-based pipelines triggered by events such as a new email, a form submission or a timer. Their strengths are broad API integration, event-based execution, visual construction and ease of use for non-programmers. Their limitation is that all logic is rule-based: branching is expressed through static conditions, and the system cannot reason about ambiguous input, adapt to a state it was not programmed for, or choose between competing actions. As task complexity grows, these platforms need increasingly elaborate rule trees that remain brittle outside the cases their authors anticipated [25].

2.2 SINGLE-AGENT REASONING AND TOOL USE

2.2.1 ReAct

Yao et al. [1] interleave free-text reasoning traces with discrete actions, so that reasoning can update a plan and decide when to query an external source, while actions return observations that feed later reasoning. On ALFWorld and WebShop the method outperforms imitation and reinforcement learning baselines by 34% and 10% absolute success. *Limitation:* the loop is a single prompt-driven process with no persistence; if the process dies the trajectory is lost, and the same input can take a different path on a second run. *Relevance:* ReAct is the execution model of a single agent node; n9n wraps such a node in a step record so that its inputs, outputs and chosen branch are stored like any other step.

2.2.2 Toolformer

Schick et al. [2] train a model, in a self-supervised way from a few demonstrations per API, to decide which tool to call, when, and with what arguments, covering a calculator,

question answering, search engines, translation and a calendar. *Limitation:* the tool set is fixed at training time and there is no mechanism for recording or retrying a call that has side effects. *Relevance:* tool calling is how an agent node would invoke n9n's HTTP and email actions; the platform, not the model, records each attempt.

2.2.3 Plan-and-Solve Prompting

Wang et al. [3] first ask the model for a plan that divides the task into subtasks and then for their execution, reducing the missing-step errors of zero-shot chain-of-thought across ten datasets. *Limitation:* the plan is free text and is not validated against an executable schema. *Relevance:* the n9n workflow builder asks a model for a plan in the form of a graph and validates that graph with the same code that guards publishing.

2.2.4 Tree of Thoughts

Yao et al. [4] treat each reasoning step as a node in a search tree that the model explores, evaluates and backtracks over; on the Game of 24, GPT-4 with chain-of-thought solved 4% of instances and Tree of Thoughts 74%. *Limitation:* search multiplies model calls and cost. *Relevance:* search over candidate graphs is a possible extension of the builder's repair loop, but its cost argues for bounding it, as n9n does with a fixed number of repair rounds.

2.2.5 Reflexion

Shinn et al. [5] reinforce agents through linguistic feedback rather than weight updates: the agent reflects on a failed trial, stores the reflection in an episodic memory and uses it in the next trial, reaching 91% pass@1 on HumanEval. *Limitation:* the method assumes trials can be repeated freely, which is false for workflows with side effects such as sending email. *Relevance:* the n9n failure explainer is a one-shot, human-reviewed form of reflection, and the platform forbids automatic retries of non-idempotent steps.

2.2.6 Self-Ask and the Compositionality Gap

Press et al. [6] define the compositionality gap, the fraction of compound questions a model fails although it answers each sub-question, and show that it does not shrink with model size; self-ask prompting, in which the model asks and answers follow-up questions, narrows it. *Relevance:* the result motivates decomposing a workflow into explicit nodes rather than asking one agent to do everything.

2.2.7 CodeAct

Wang et al. [7] use executable Python code as an agent's action space, reporting up to 20% higher success than JSON or text actions across 17 models. *Limitation:* arbitrary code execution needs sandboxing and makes the effect of an action hard to audit. *Relevance:* n9n plans a sandboxed WebAssembly code node with no system capabilities and fuel and memory limits, disabled by default.

2.2.8 Gorilla

Patil et al. [8] fine-tune a LLaMA model to write API calls and pair it with a document retriever so that it adapts to documentation changes at test time. *Limitation:* hallucinated calls are reduced but still possible. *Relevance:* n9n gives the model the node catalogue with its configuration, input and output schemas, and rejects any generated graph that fails validation.

2.3 MULTI-AGENT AND ORCHESTRATION FRAMEWORKS

2.3.1 AutoGen

Wu et al. [9] present an open-source framework in which customisable, conversable agents, backed by models, humans or tools, converse to complete a task. *Limitation:* conversation state lives in process memory; there is no durable log, replay or deduplication of side effects. *Relevance:* AutoGen's human-in-the-loop agent corresponds to n9n's planned approval node, which pauses a run in the database until a person decides.

2.3.2 CAMEL, MetaGPT and HuggingGPT

CAMEL [10] studies role-playing between communicative agents guided by inception prompts. MetaGPT [11] encodes standard operating procedures as prompt sequences and assigns agents roles in an assembly line, so that intermediate results are verified and cascading hallucinations are reduced. HuggingGPT [12] uses a chat model as a controller that plans tasks and selects models from a hub by their descriptions. *Limitation:* all three keep procedures in prompts and trust the controller's plan. *Relevance:* role separation informs how an SEO content agent and a verification agent could be split; MetaGPT's verified intermediates match n9n's rule that every agent output passes a deterministic validator; HuggingGPT is an early example of planning over a catalogue of tools, which is what the n9n builder does over node types.

2.3.3 Voyager

Wang et al. [13] sustain open-ended behaviour in Minecraft with an automatic curriculum, a library of skills stored as executable code, and iterative prompting driven by execution errors. *Limitation:* the setting is a game with no side effects outside it. *Relevance:* a published workflow version is a reusable skill; n9n stores versions immutably so a skill cannot change under a running job.

2.3.4 n8n and LangGraph

n8n [14] is an open-source node-based automation platform with hundreds of integrations and an AI agent node, reported to reach high workflow reliability in enterprise deployments [25]. LangGraph [15] orchestrates stateful agent graphs with checkpointing and human-in-the-loop interrupts for long-running business processes [27]. *Limitation:* both allow cycles and arbitrary code in nodes, which weakens validation at publish time, and neither documents a per-request attempt record or classifies an interrupted side effect as uncertain. *Relevance:* they are the systems closest to n9n, which deliberately restricts a workflow to a validated directed acyclic graph with one trigger.

2.4 EVALUATION OF AGENTIC SYSTEMS

WebArena [16] provides self-hosted, functional websites with tasks graded on functional correctness; the best GPT-4 agent reaches 14.41% end-to-end success against 78.24% for humans. AgentBench [17] evaluates models as agents in eight environments and attributes failures to poor long-term reasoning, decision-making and instruction following. GAIA [18] poses 466 real-world assistant questions on which humans score 92% and GPT-4 with plugins 15%. τ-bench [19] scores a tool-using agent by the final database state and introduces pass^k, the probability of succeeding in all of k repeated trials; GPT-4o passes fewer than half the tasks and has pass^8 below 25% in its retail domain. SWE-agent [20] shows that an agent-computer interface designed for a model, with constrained commands, changes agent behaviour, reaching 12.5% pass@1 on SWE-bench.

Together these benchmarks show that agentic methods outperform naive prompting but remain far from reliable autonomous execution on open-ended tasks. For n9n this has three consequences: an agent must not be given unsupervised control of side-effecting steps; deterministic checks must stay deterministic; and agentic components must be measured over repeated trials, for which n9n adopts pass^k.

2.5 SEO AND GENERATIVE ENGINE OPTIMISATION

Aggarwal et al. [21] formalise generative engines, propose a black-box framework for raising a source's visibility in generated answers and release GEO-bench; the best strategies raise visibility by up to 40%, but efficacy varies by domain. *Limitation:* the work optimises content for visibility and does not address auditing a site's technical health. *Relevance:* it defines the content-side SEO tasks that n9n's agent nodes would perform after the deterministic technical checks pass.

2.6 SURVEYS OF LLM AGENTS

Wang et al. [22] propose a unified construction framework for LLM agents; Xi et al. [23] frame agents as brain, perception and action across single-agent, multi-agent and human-agent settings; Masterman et al. [24] compare single- and multi-agent architectures for reasoning, planning and tool calling. All three list reliability, evaluation and safe tool use as open problems, which is the gap this project addresses from the systems side. Industry accounts [26] describe the same shift from rule-based to agentic workflow automation.

2.7 COMPARATIVE ANALYSIS

Table 2.1 contrasts the two paradigms. Table 2.2 compares the reviewed works on the properties a production workflow substrate needs: whether execution state is durable, whether a run can be replayed from a step, and whether side effects are protected against duplication. The columns are judged from each work's own description; *no* means the work does not address the property, which is not a criticism of its goals.

Table 2.1 Rule-based automation versus agentic orchestration

Dimension	**Rule-based tools (Zapier, Make, n8n, Node-RED)**	**Agentic AI orchestration**
Control logic	Fixed if-this-then-that rules	Reasoning and planning driven by an LLM
Adaptability	Cannot handle unforeseen states	Can revise plans from context and feedback
Coordination	Single linear pipeline	Multiple cooperating agents (e.g. AutoGen)
Decision-making	None; execution only	Chain-of-thought or tree-of-thought reasoning
Reproducibility	High; same input gives same path	Low; same input can take a different path

Dimension	Rule-based tools (Zapier, Make, n8n, Node-RED)	Agentic AI orchestration
Evaluation	Task completion logs	Benchmarks (AgentBench, WebArena, GAIA, τ-bench)

Table 2.2 Comparison of reviewed works on substrate properties

Work	Strand	Main contribution	Durable state	Replay	Side-effect safety	Use in n9n
ReAct [1]	Single agent	Interleaved reasoning and acting	no	no	no	Execution model of an agent node
Toolformer [2]	Single agent	Self-supervised tool calling	no	no	no	Tool calls become HTTP and email steps
Plan-and-Solve [3]	Single agent	Plan before execution	no	no	no	Builder plans a graph, then it is validated
Tree of Thoughts [4]	Single agent	Search over reasoning steps	no	no	no	Bounded repair rounds
Reflexion [5]	Single agent	Verbal self-reflection	In-process memory	no	no	One-shot explainer, human reviewed
Self-Ask [6]	Single agent	Decomposition into sub-questions	no	no	no	Explicit nodes, not one agent
CodeAct [7]	Single agent	Code as the action space	no	no	no	Sandboxed code node, off by default
Gorilla [8]	Single agent	API-call synthesis with retrieval	no	no	no	Catalogue schemas in the builder prompt
AutoGen [9]	Multi-agent	Conversable agents incl. humans	no	no	no	Approval node as human agent
CAMEL [10]	Multi-agent	Role-playing agents	no	no	no	Separate content and verification roles
MetaGPT [11]	Multi-agent	SOPs, verified intermediates	no	no	no	Validator between agent and use
HuggingGPT [12]	Multi-agent	LLM controller over a model hub	no	no	no	Planning over a node catalogue
Voyager [13]	Multi-agent	Skill library, curriculum	Skill library	no	no	Immutable versions as skills
n8n [14]	Platform	Node-based automation, AI node	Yes	Retry from failed node	no	Closest product; n9n restricts graphs
LangGraph [15]	Platform	Stateful graphs, checkpoints	Yes	From checkpoint	no	n9n adds the uncertain state
WebArena [16]	Evaluation	Functional web tasks	no	no	no	Bounds autonomy of agent nodes
AgentBench [17]	Evaluation	Eight-environment benchmark	no	no	no	Model chosen by configuration
GAIA [18]	Evaluation	Real assistant questions	no	no	no	Keep deterministic checks deterministic
τ-bench [19]	Evaluation	State-based scoring, pass^k	no	no	no	Metric for the agentic layer

Work	Strand	Main contribution	Durable state	Replay	Side-effect safety	Use in n9n
SWE-agent [20]	Evaluation	Agent-computer interface	no	no	no	Narrow, typed node interface
GEO [21]	SEO	Visibility in generative engines	no	no	no	Content tasks for agent nodes
Surveys [22]-[24]	Survey	Design space, open problems	no	no	no	Framing of the gap

2.8 SUMMARY OF REVIEWED LITERATURE

Table 2.3 Year-wise summary of key reviewed works

Year	Authors / work	Key contribution
2023	Yao et al., ReAct [1]	Reason-act-observe loop; basis of most agent executors
2023	Schick et al., Toolformer [2]	Model learns which API to call, when and with what arguments
2023	Wang et al., Plan-and-Solve [3]	Explicit plan before execution reduces missing-step errors
2023	Yao et al., Tree of Thoughts [4]	Searchable tree of reasoning steps with backtracking
2023	Shinn et al., Reflexion [5]	Verbal reflection stored across trials
2023	Wu et al., AutoGen [9]	Multi-agent conversation framework mixing LLMs, humans and tools
2023	Wang et al., Voyager [13]	Skill library, automatic curriculum, feedback-driven code
2024	Zhou et al., WebArena [16]	Reproducible web environment; agents far below humans
2024	Liu et al., AgentBench [17]	Eight environments; reasoning and decision gap between models
2023	Mialon et al., GAIA [18]	Real-world assistant questions needing tools and browsing
2024	Yao et al., τ-bench [19]	Database-state scoring and the pass^k reliability metric
2024	Aggarwal et al., GEO [21]	Optimising visibility in generative search engines
2025	Venkiteela, n8n [25]	Open-source enterprise workflow engine with AI orchestration
2025	Chirra, agentic AI [26]	How agentic AI is changing workflow automation
2026	Pearson et al., LangGraph [27]	Graph-based orchestration of long-running stateful processes

2.9 RESEARCH GAP

The reviewed literature gives a mature account of how a single model reasons and calls tools, several ways to coordinate many agents, and increasingly realistic benchmarks that show those agents succeeding on a minority of open-ended tasks. What it does not give is the systems layer that a production workflow needs around those agents:

- **No durable, replayable execution record.** ReAct-style loops and conversation frameworks keep state in process memory. LangGraph forks from a checkpoint and n8n

retries from the failed node; neither records the replay as a linked run with validated lineage, lets the user replay against an edited draft, or classifies an interrupted side effect.

- **No treatment of interrupted side effects.** None of the works distinguishes a step that may be retried from one whose effect is unknown after a crash. Reflexion and Tree of Thoughts assume trials are free to repeat.
- **Validation of agent output is left to the model.** Plan-and-Solve and HuggingGPT trust the generated plan; Gorilla reduces but does not eliminate hallucinated calls. No work routes a generated artefact through the same validator that guards human-authored ones.
- **Deterministic and agentic steps are not separated by design.** Benchmarks show agents failing tasks a rule would solve, yet frameworks let an agent own the whole task.
- **SEO is treated as content optimisation only.** No reviewed work combines deterministic technical audits with model-based content assessment in one automated workflow.
- **Reliability is rarely measured over repeated trials.** τ-bench's pass^k is the exception; platform papers report single-run success.

n9n addresses the first four gaps with an implemented substrate, and the last two with a specified design and an evaluation plan.

CHAPTER 3 – METHODOLOGY

This chapter describes how n9n was designed and built: the development approach, the requirements, the three-layer architecture, the workflow model and execution engine, the database and API design, the implementation of each component, the deployments, the design of the agentic and SEO layers, and the testing strategy.

3.1 DEVELOPMENT APPROACH

The project followed an incremental approach in four stages. First, the literature review fixed the properties the substrate had to provide. Second, the API contract (`contracts/openapi.yaml`) and the node catalogue (`contracts/nodes.md`) were written before the code, so that the Go API, the Rust worker and both clients were built against one specification. Third, the components were implemented feature by feature, each with automated tests, under version control (80 commits between 24 September and 30 October 2026, with changes merged through pull requests). Fourth, the substrate was measured with a reproducible harness, and the platform was deployed locally and on Cloudflare.

3.2 REQUIREMENTS

Table 3.1 Functional requirements

ID	Requirement
FR1	A user can register, sign in and sign out on the web and on mobile.
FR2	A user can build a workflow on a visual canvas from a catalogue of trigger and action nodes, and save it as a draft.
FR3	Publishing validates the graph and stores it as an immutable, numbered version.
FR4	A workflow can be started manually, by an authenticated webhook, on a schedule, or by a new Gmail message.
FR5	A draft can be test-run without publishing it.
FR6	Every run records each step's status, input, output, branch, attempt number and timing, and every HTTP attempt.
FR7	A finished run can be replayed from any node, optionally with edited input or against the current draft.
FR8	Gmail credentials are connected through Google OAuth and stored encrypted; secrets are never returned to clients.
FR9	A run can be cancelled, and the run history can be inspected step by step.

Table 3.2 Non-functional requirements

ID	Requirement
NFR1	Durability: run and step state survive a crash of the worker or the API.
NFR2	Side-effect safety: an interrupted non-idempotent action is never repeated automatically.
NFR3	Scalability: several workers can share one database without claiming the same run twice.
NFR4	Security: per-IP limits on sign-in, encrypted secrets, outbound requests only to public addresses by default.
NFR5	Portability: the whole stack starts with one Docker Compose command and no manual secret setup.
NFR6	Consistency: the Go and TypeScript implementations of the API behave identically, checked automatically.

3.3 SYSTEM ARCHITECTURE

n9n is organised in three layers (Fig. 3.1). The deterministic substrate stores, validates, versions and executes workflows and is implemented. The agentic layer adds model-driven construction and diagnosis of workflows and, later, agent nodes inside them; it is specified and switched off by configuration. The SEO layer is an application of both: technical checks are ordinary deterministic nodes, and content assessment is an agent node.

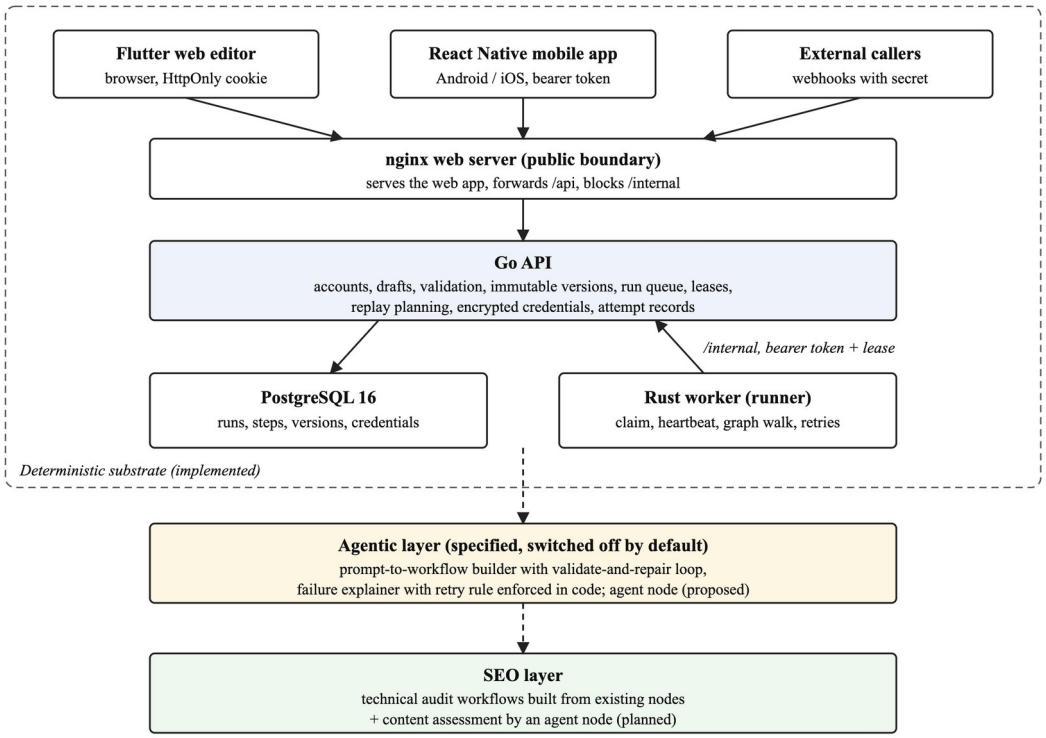


Fig. 3.1 Three-layer architecture of n9n

Fig. 3.2 shows the containers of the substrate and who talks to whom. Only the nginx web server is published to the network. It serves the compiled Flutter web app and forwards `/api/*` to the Go API, and it blocks the `/internal` routes that the worker uses. The Go API is the only component that reads and writes PostgreSQL. The Rust worker has no database access: it claims jobs, reports steps and renews its lease through the internal API with a bearer token. External services are Google OAuth and the Gmail API for email triggers and sending, and any public HTTP endpoint called by an HTTP request node.

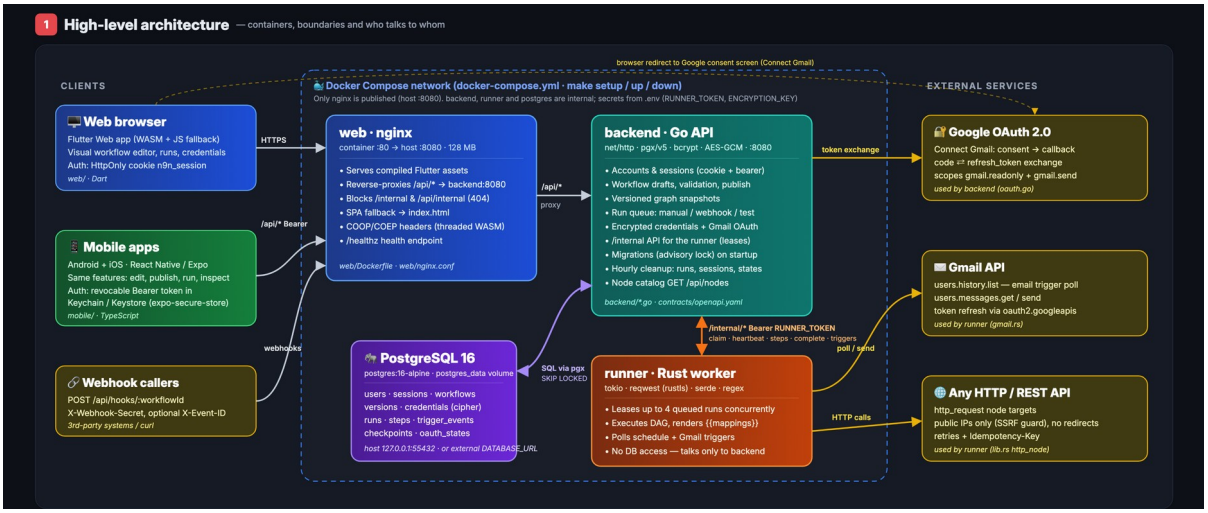


Fig. 3.2 Containers, boundaries and data flows of the deterministic substrate (from the project's architecture diagram; password hashing has since changed from bcrypt to PBKDF2-SHA256, Section 3.8)

3.4 WORKFLOW MODEL

A workflow is a directed acyclic graph of typed nodes. Publishing validates that the graph has exactly one trigger, that every action has at most one incoming edge, that every node is reachable from the trigger, that there is no cycle, and that every `{{nodes.ID.field}}` mapping refers to an ancestor node. Actions receive the output of their single incoming edge as `input`; a string made entirely of one mapping keeps the mapped JSON type, and a missing field fails the step. The node catalogue (version 2, served by `GET /api/nodes`) has eight types, listed in Table 3.3. Only the condition node has `true` and `false` output ports; all others output on `out`.

Table 3.3 Node catalogue (version 2)

Node	Kind	Behaviour
manual_trigger	Trigger	Run now with a JSON input

Node	Kind	Behaviour
webhook_trigger	Trigger	POST /api/hooks/{workflow_id} with a secret header; events deduplicated by ID
schedule_trigger	Trigger	Every N minutes (N ≥ 1), polled by the worker
email_trigger	Trigger	New Gmail message matching a query, polled at least every 30 s
set_fields	Action	Merges rendered fields into the input
http_request	Action	Method, URL, headers, body; returns status, headers and body
condition	Action	Compares two values (equals, contains, greater than, exists...) and routes to true or false
send_email	Action	Sends a message through the Gmail API

Publishing snapshots the graph as an immutable numbered version, so a draft edit never changes a queued or running job. A draft can also be executed as a test run against a snapshot stored on the run record, without publishing. Fig. 3.3 shows the life of a workflow from the editor to a completed run.

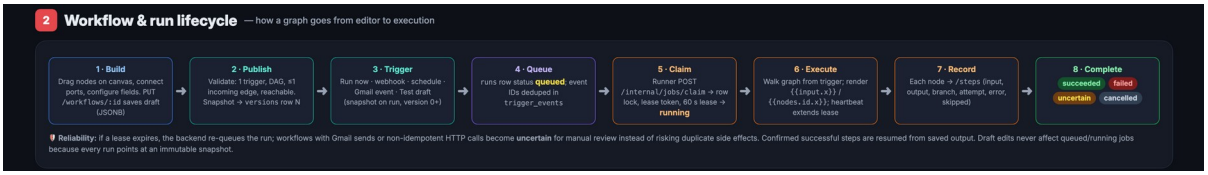


Fig. 3.3 Workflow and run lifecycle, from building a draft to a terminal run status

3.5 EXECUTION ENGINE

3.5.1 Durable Queue and Leases

Runs are rows in PostgreSQL. A worker claims the oldest queued run with a row lock and `SKIP LOCKED`, so several workers can share one database, and receives a 60-second lease that it extends every 10 seconds by heartbeat. Every step reports its status, input, output, branch, attempt number and timing to the API, so the database holds the full execution trace. If a lease expires, the next claim examines the steps that were running or failed. If any of them belongs to a send-email node, or to an HTTP node whose method is not GET or HEAD and that has no idempotency key, the run is marked *uncertain* with the reason recorded; otherwise it is re-queued and resumed, reusing every step that already succeeded. A cancelled run fences the old lease so a late worker cannot write to it.

3.5.2 Attempts, Limits and Isolation

GET, HEAD and idempotent HTTP requests are attempted up to three times on a 5xx response or a transport failure; every request, including retries, is recorded as an attempt with status, error and duration. Redirects are not followed, so a 301 is reported as such. Destinations must resolve to public addresses unless the operator allows private ones, which protects internal services from server-side request forgery. Step outputs are capped at 256 KiB, JSON nesting at 32 levels and public request bodies at 1 MiB.

3.5.3 Replay

A finished run can be replayed from any node, with an optional input override, against its original graph or the current draft. The API walks the graph from the trigger the way the worker will: conditions on the path to the chosen node are forced onto that path, other conditions keep their recorded branch, and only reached nodes that succeeded in the parent are copied as succeeded steps. A node that never completed in the parent is rejected rather than silently re-sent. The replay is a new run of kind `replay` linked to its parent, so the lineage can be queried.

3.6 DATABASE DESIGN

The schema has twelve tables, created and upgraded by versioned migrations that the API applies at start-up under a PostgreSQL advisory lock. Table 3.4 lists them and Fig. 3.4 shows the original design. Later migrations added the run kind (`normal`, `test`, `replay`), the parent run and resume node used by replay, per-step start and finish times, and the `step_attempts` table. A check constraint and a trigger enforce that test and replay runs carry a graph snapshot while published runs reference a version. Terminal runs are deleted after `RUN_RETENTION_DAYS` (30 by default); webhook event IDs remain as deduplication tombstones.

Table 3.4 Database tables

Table	Purpose
users	Accounts: email and PBKDF2-SHA256 password hash
sessions	Hashed session tokens for browser cookies and mobile bearer tokens
oauth_states	Single-use state values for the Google OAuth flow
credentials	Gmail credentials; secrets encrypted with AES-256-GCM
workflows	Name, draft graph (JSONB), published version, active flag
versions	Immutable published graph snapshots
runs	Run queue and history: status, lease, kind, parent run, snapshot

Table	Purpose
steps	One row per node execution: input, output, branch, attempt, timing
step_attempts	Every HTTP request made by a step, including retries
trigger_events	Webhook and Gmail event IDs for deduplication
checkpoints	Gmail history cursors and schedule state
schema_migrations	Applied migration versions

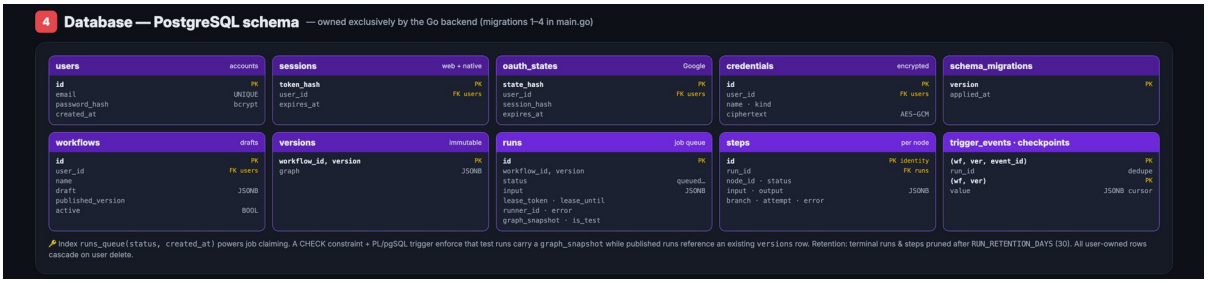


Fig. 3.4 Database schema as first designed (later migrations add step_attempts and the replay columns described above)

3.7 API DESIGN

The public API is JSON over HTTP under `/api`, described in an OpenAPI contract. Table 3.5 lists the main routes. Browser sessions use an HttpOnly cookie; mobile clients use revocable bearer tokens issued by the `/auth/mobile/*` routes and stored in the iOS Keychain or Android Keystore. The worker uses the `/internal/*` routes with the `RUNNER_TOKEN` bearer token: claim, heartbeat, steps, attempts, complete, and trigger polling and checkpoints.

Table 3.5 Main public API routes

Route	Purpose
POST /auth/register, /auth/login, /auth/logout; GET /auth/me	Browser accounts and cookie sessions
POST /auth/mobile/register, /auth/mobile/login	Native sign-in with bearer tokens
GET /oauth/google/start, /oauth/google/callback	Connect a Gmail account
GET /nodes, /features	Node catalogue and feature flags
GET, POST /workflows; GET, PUT, DELETE /workflows/{id}	Create, list, edit and delete workflows
POST /workflows/{id}/publish, /activate, /run, /test	Publish a version, enable triggers, run, test a draft
GET /runs, /runs/{id}; POST /runs/{id}/cancel, /runs/{id}/replay	Run history, inspection, cancel and replay
GET, POST /credentials; DELETE /credentials/{id}	Manage credentials (metadata only)
POST /hooks/{workflow_id}	Webhook trigger with secret and optional event ID

3.8 IMPLEMENTATION OF THE COMPONENTS

Fig. 3.5 summarises the internal structure of each component. The main implementation decisions are described below.

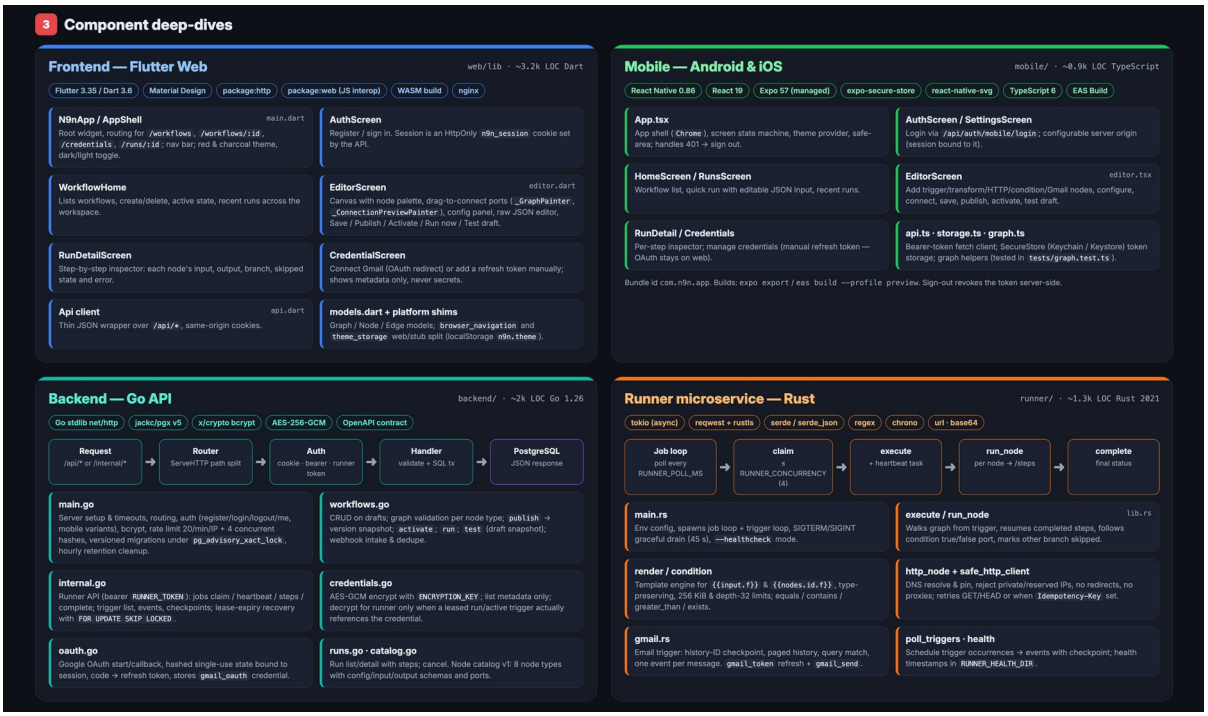


Fig. 3.5 Internal structure of the web, mobile, API and worker components

3.8.1 Go API

The API (about 2,700 lines of Go 1.26, excluding tests) uses the standard `net/http` server and the `pgx` PostgreSQL driver. It implements accounts and sessions, workflow drafts, graph validation, publishing, the run queue with leases, replay planning, credential encryption, the Google OAuth flow, webhook intake with deduplication, and an hourly clean-up of expired runs, sessions and OAuth states. Passwords are hashed with PBKDF2-SHA256 (100,000 iterations), a format shared with the TypeScript API so that an account works on either. Sign-in is limited to 20 attempts per source IP per minute and four concurrent password checks.

3.8.2 Rust Worker

The worker (about 1,500 lines of Rust on the `tokio` runtime) polls the API for queued runs and executes up to `RUNNER_CONCURRENCY` runs at once (4 by default). For each run it walks the graph from the trigger, renders `{{input.x}}` and `{{nodes.id.x}}` mappings, follows the true or false port of each condition and marks the other branch skipped, executes

HTTP and Gmail actions, and reports every step and attempt. It also polls schedule and Gmail triggers. HTTP requests resolve and pin the destination address, reject private and reserved addresses, never follow redirects, and retry only GET, HEAD and requests with an idempotency key. On shutdown the worker drains for up to 45 seconds.

3.8.3 Flutter Web Editor

The web editor (about 3,900 lines of Dart, Flutter 3.35, compiled to WebAssembly) provides sign-in, the workflow list, the canvas editor with a node library, port-to-port connections, a configuration panel and undo and redo, the run inspector with a timeline, the run canvas and a step-by-step view, and the credentials page. It calls `/api` on its own origin with an HttpOnly cookie and supports light and dark themes.

3.8.4 React Native Mobile App

The mobile app (React Native 0.86 with Expo, TypeScript) offers the same core functions on Android and iOS: sign-in with a bearer token held in secure storage, the workflow list, an editor for the initial node types, publishing, activation, manual runs with editable JSON input, draft tests and the per-step run inspector. The server address is configurable in the app, and changing it clears the saved session.

3.9 DEPLOYMENT

3.9.1 Local Deployment with Docker Compose

One `docker-compose.yml` file builds and runs the whole stack: the nginx web server with the compiled Flutter app (port 8080), the Go API, the Rust worker and PostgreSQL 16. A one-off `secrets` service generates a random `ENCRYPTION_KEY` and `RUNNER_TOKEN` on the first start and keeps them in a Docker volume, so a new machine needs no configuration file; values in an optional `.env` file take precedence. The API starts only after PostgreSQL is healthy, and nginx resolves the API address through Docker's DNS on each request so that a restarted container is found again. The command `docker compose up --build -d --wait` starts everything and waits until all services report healthy.

3.9.2 Cloud Deployment on Cloudflare Workers

The public deployment runs on the Cloudflare Workers free plan at `https://n9n.walrus.codes`. The Flutter app is served as static assets, and the API is a TypeScript port of the Go API running as a Worker, connected to a managed PostgreSQL

database through Cloudflare Hyperdrive. Both implementations are kept in sync: every change to API behaviour lands in both, a script fails the build if their schema migrations differ, and a parity test suite runs the same requests against both servers in continuous integration. A GitHub Actions workflow deploys the Worker after the CI checks pass on the main branch. The free plan limits CPU time to 10 ms per request; measured on the live Worker, the median request used 4.2 ms and sign-in about 36 ms, which Cloudflare tolerates occasionally. The Rust worker is not hosted on Cloudflare yet.

3.10 DESIGN OF THE AGENTIC LAYER

All model calls go through one client in the API, are off unless an API key is configured, are rate-limited per user (30 requests per hour by default) and are logged with token counts. Workflow graphs may be sent to the model; run inputs and outputs only if the operator enables it and the user confirms. Credential values, webhook secrets and HTTP header values are redacted before any prompt is built, with a unit test that fails if a known secret appears in an outgoing request.

- **Prompt-to-workflow builder.** Given a sentence, the model receives the node catalogue with its schemas, the user's credential names and the mapping syntax, and returns a graph in the draft format. The API runs the publish validator on the proposal; on failure it returns the errors to the model and asks for a corrected graph, up to three rounds. Only a validated graph reaches the editor, as an unsaved draft the user can test. This is Plan-and-Solve with a code validator in place of the model's own judgement.
- **Failure explainer.** For a failed or uncertain run, the model receives the graph, the failing step, prior steps subject to the data policy, the recorded attempts and the documentation of the node types involved, and returns a summary, a likely cause, a `safe_to_retry` verdict and an optional fix. The verdict is overridden in code to *no* for any uncertain run whose unsafe node lacks an idempotency key. A proposed fix can be tested through replay against the draft before publishing.
- **Agent node (proposed).** An agent node would run a ReAct loop whose tools are the existing node actions, with its reasoning trace stored as the step output and its tool calls recorded as attempts. Because it may call side-effecting tools, it is classified unsafe for lease recovery, like send-email.

Table 3.6 Mapping of n9n components to the literature

Component	Basis	How n9n extends it
Agent node (proposed)	ReAct, Toolformer	Trace and tool calls stored as step output and attempts; unsafe for lease recovery
Builder	Plan-and-Solve, Gorilla, HuggingGPT	Plan is a graph checked by the publish validator; up to three repair rounds
Explainer	Reflexion	One reflection, human reviewed; retry verdict enforced in code
Approval node (planned)	AutoGen	Pause persists in the database, not in process memory
Versions	Voyager	Immutable and numbered; a draft edit cannot alter a running job
Replay	LangGraph	From any node, with input override, reusing recorded upstream outputs
Uncertain state	none (gap)	Interrupted non-idempotent step is surfaced, never retried automatically
Graph restrictions	n8n	One trigger, one input per action, no cycles, validated at publish
Reliability metric	τ-bench	pass^k planned for the agentic components
SEO audit	GEO	Deterministic technical checks precede model content assessment

3.11 DESIGN OF THE SEO LAYER

Technical SEO auditing decomposes into checks with exact answers: a page is reachable or not, returns 200 or not, redirects or not, has a title tag or not, and serves `robots.txt` and a sitemap or not. Fig. 3.6 shows the audit workflow built from existing nodes only: a trigger receives a URL, an HTTP node fetches it, a condition on the body separates pages without a title, and set-fields nodes record a verdict. A page that is unreachable or answers with a non-2xx status fails the fetch step, and the status is read from the recorded attempt. Because the HTTP node records attempts, a page that fails twice and then succeeds is reported with three attempts rather than as a failure. Content assessment (relevance to a target query, intent match, readability, and the GEO visibility strategies of [21]) is where an agent node is used, after the deterministic checks pass, so that a model never decides whether a page exists.

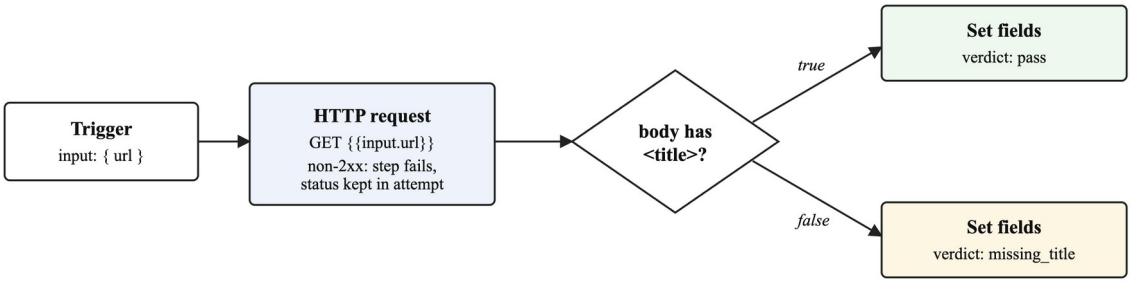


Fig. 3.6 Technical SEO audit workflow built from existing n9n nodes

3.12 TESTING STRATEGY

Testing was done at four levels. Unit and integration tests cover the Go API (35 test functions, run against an isolated PostgreSQL schema with the race detector, including concurrent claiming, lease recovery and replay), the Rust worker (25 tests: graph execution, mapping rendering, HTTP safety and retries, Gmail history), the Flutter editor and the mobile app. End-to-end tests drive the public API (registration, mappings, branches, publishing, cross-user access, webhook secrets and deduplication, private-address rejection, schedules, cancel, mobile bearer sessions, replay and restart recovery) and the browser UI through Playwright. Parity tests run the same cases against the Go and TypeScript APIs. Finally, the measurement harness in Chapter 4 exercises latency, throughput, replay, SEO auditing and fault injection. All suites run in GitHub Actions on every push.

3.13 TOOLS AND TECHNOLOGIES

Table 3.7 Tools and technologies used

Area	Technology
API	Go 1.26, net/http, pgx v5, AES-256-GCM, PBKDF2-SHA256
Worker	Rust (tokio, reqwest with rustls, serde, regex, chrono)
Database	PostgreSQL 16
Web client	Flutter 3.35 / Dart, compiled to WebAssembly, served by nginx
Mobile client	React Native 0.86, Expo, TypeScript, expo-secure-store
Cloud API	TypeScript on Cloudflare Workers, Hyperdrive, managed PostgreSQL
Integrations	Google OAuth 2.0, Gmail API, any public HTTP API
Packaging	Docker, Docker Compose, Make
Testing	go test, cargo test, flutter test, Node test runner, Playwright, Python smoke tests
CI/CD and contracts	GitHub Actions, Wrangler, OpenAPI 3

CHAPTER 4 – RESULTS AND DISCUSSION

This chapter presents the implemented system and its measured behaviour. The measurements cover the deterministic substrate only; no language model was called in any experiment, and nothing in this chapter is a claim about the quality of agentic decisions.

4.1 IMPLEMENTATION STATUS

Table 4.1 Implementation inventory

Component	Size	Tests	Executed
Go API (backend)	~2,700 lines	35 functions	Yes; all passed (suite of 34 on 23 Oct 2026, 35.8 s)
Rust worker (runner)	~1,500 lines	25	Yes; all passed (23 Oct 2026)
Flutter web editor	~3,900 lines	15	In CI; browser tests below
React Native mobile app	~1,200 lines	6	Yes; type check and 6 tests passed (30 Oct 2026)
TypeScript Worker API	~1,800 lines	Parity suite	In CI against both APIs
End-to-end API smoke tests	n/a	7 groups	Yes; passed on Docker stack and Worker (30-31 Oct 2026)
Browser tests (Playwright)	n/a	6	In CI
PostgreSQL schema	12 tables	n/a	Applied at start-up
Node catalogue	8 types (v2)	n/a	Served by GET /api/nodes

4.2 USER INTERFACE

The screenshots in this section were taken from the Docker Compose deployment on 31 October 2026, using workflows created through the public API: the seven-node branching workflow used for the latency experiment and the technical SEO audit workflow.

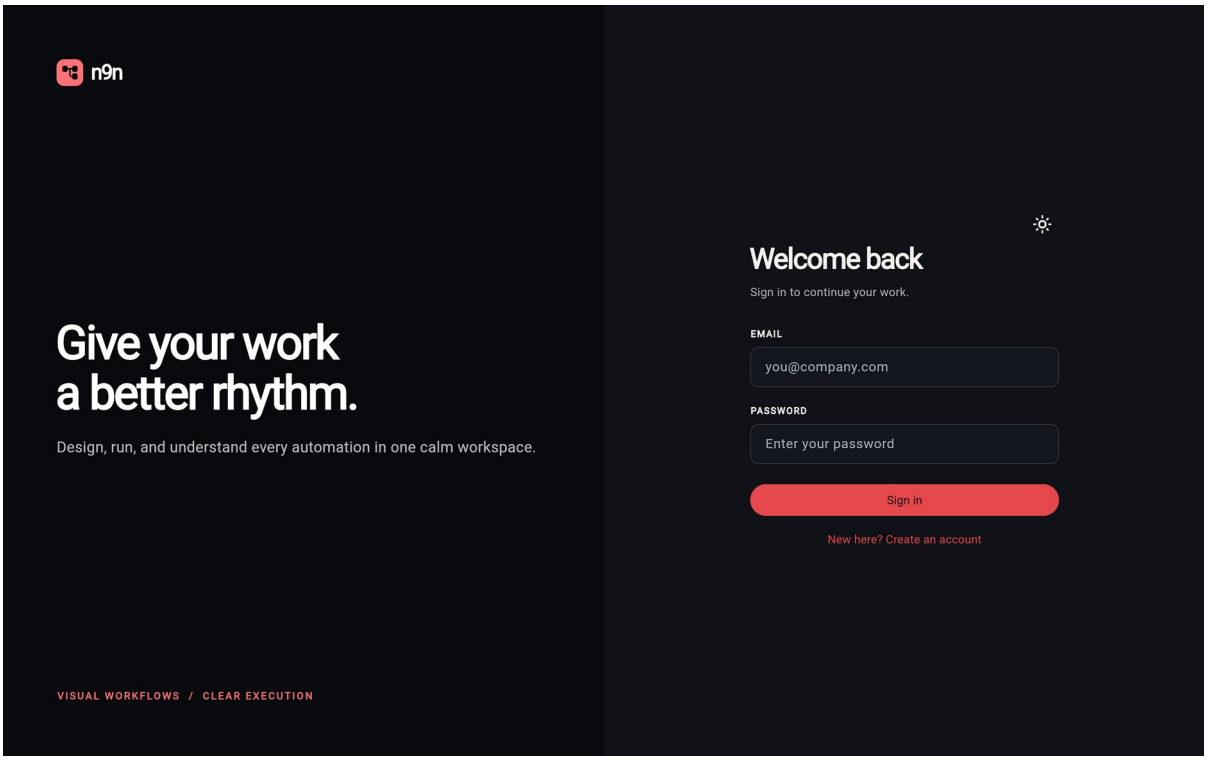


Fig. 4.1 Sign-in screen of the web editor

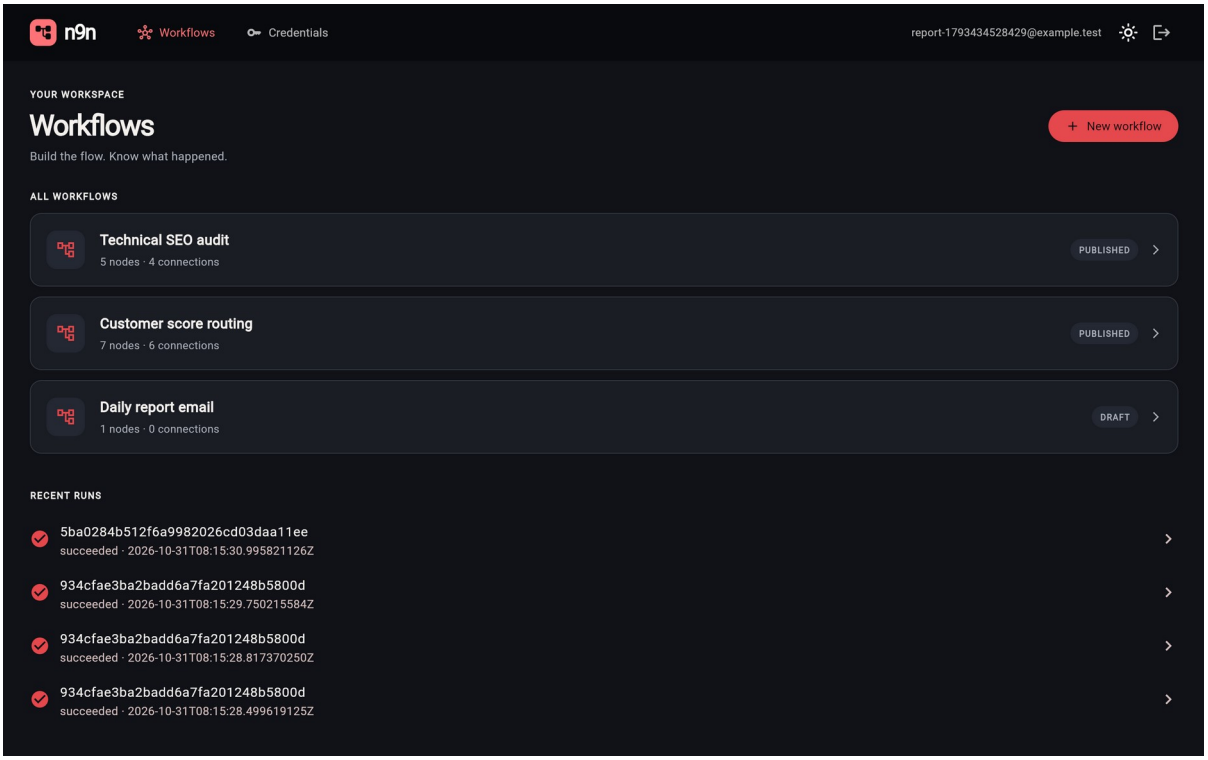


Fig. 4.2 Workflow list with published and draft workflows and recent runs

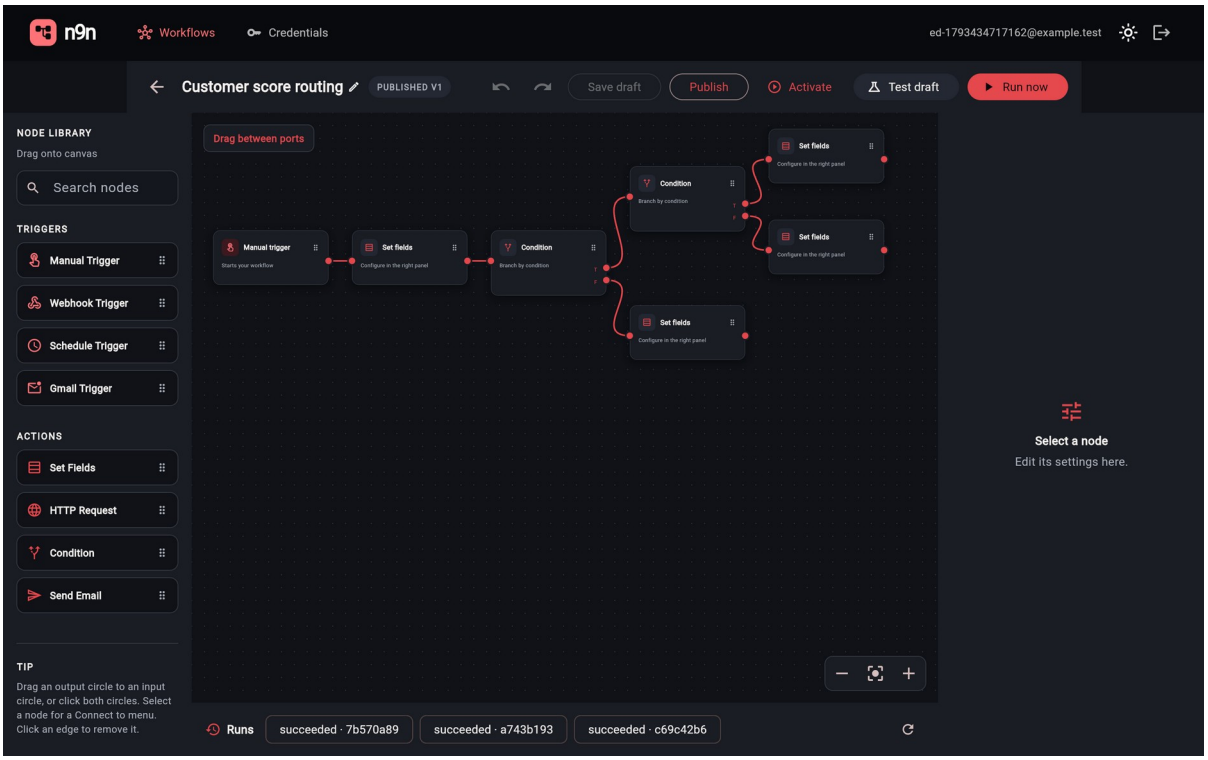


Fig. 4.3 Canvas editor showing the seven-node branching workflow, the node library and recent runs

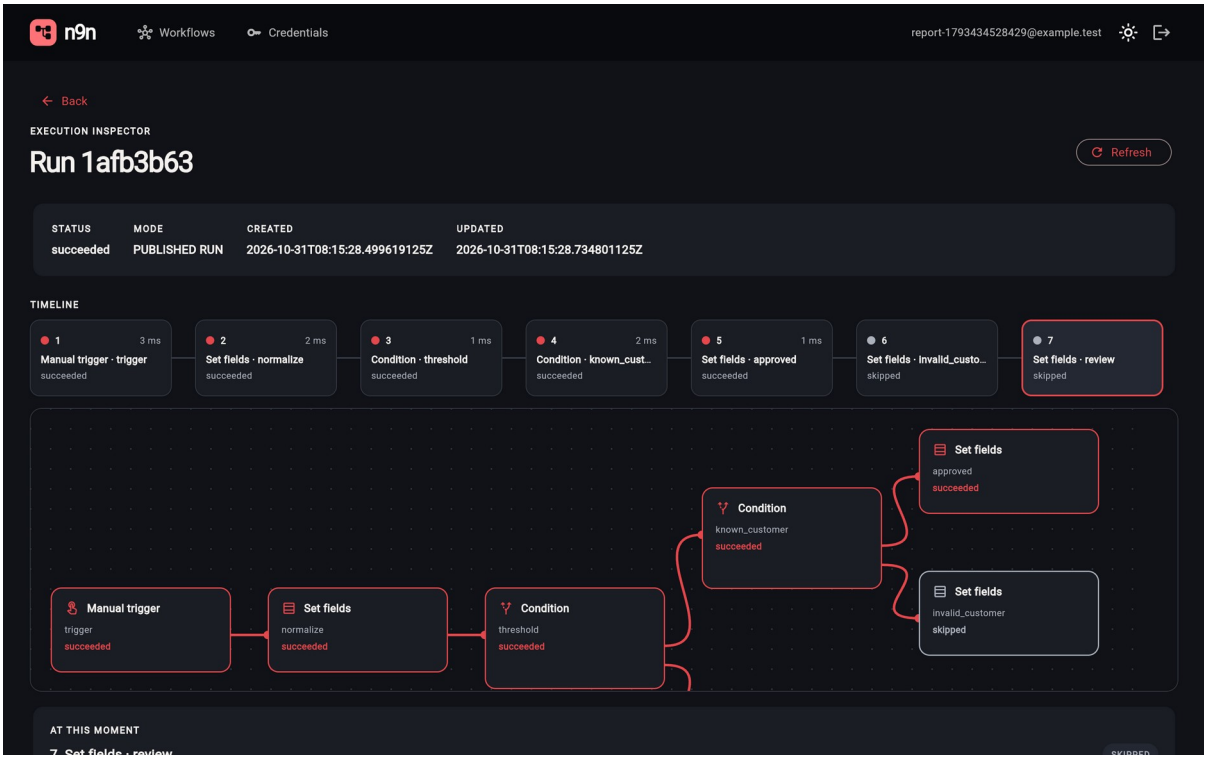


Fig. 4.4 Execution inspector for a run of the branching workflow: timeline, run canvas and per-node status, with the untaken branches marked skipped

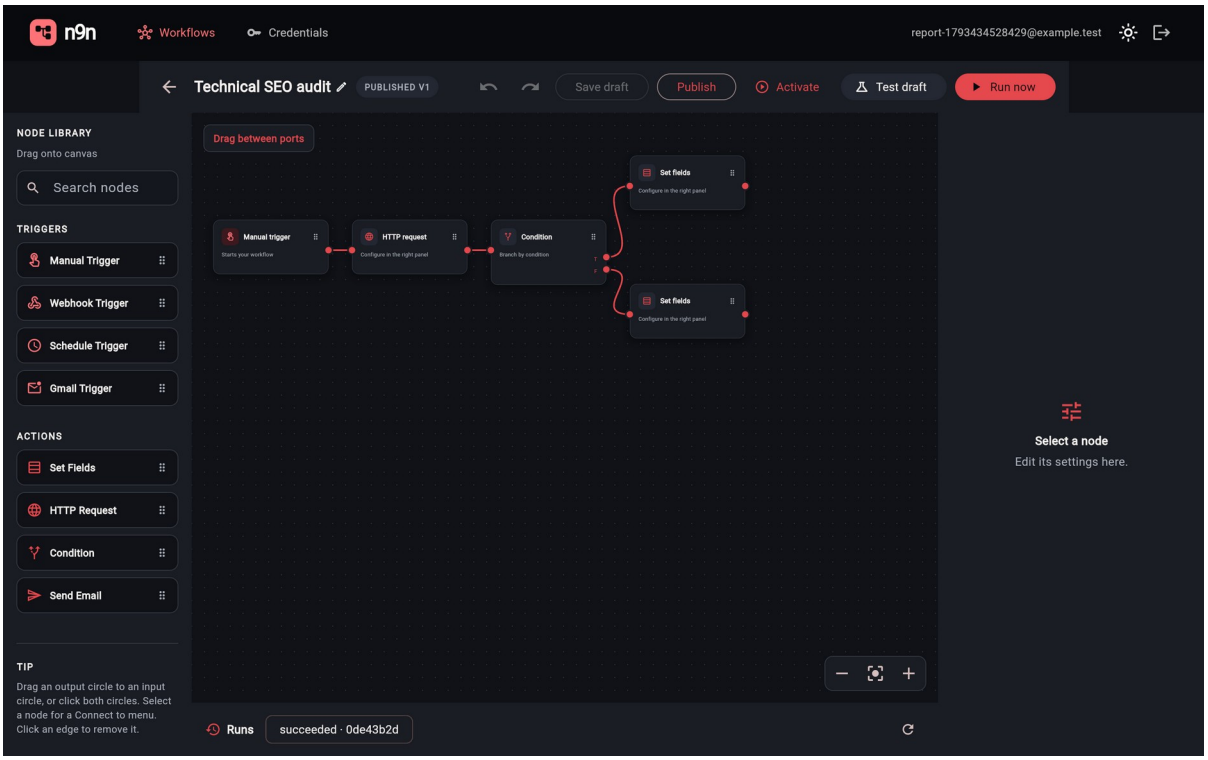


Fig. 4.5 The technical SEO audit workflow in the editor

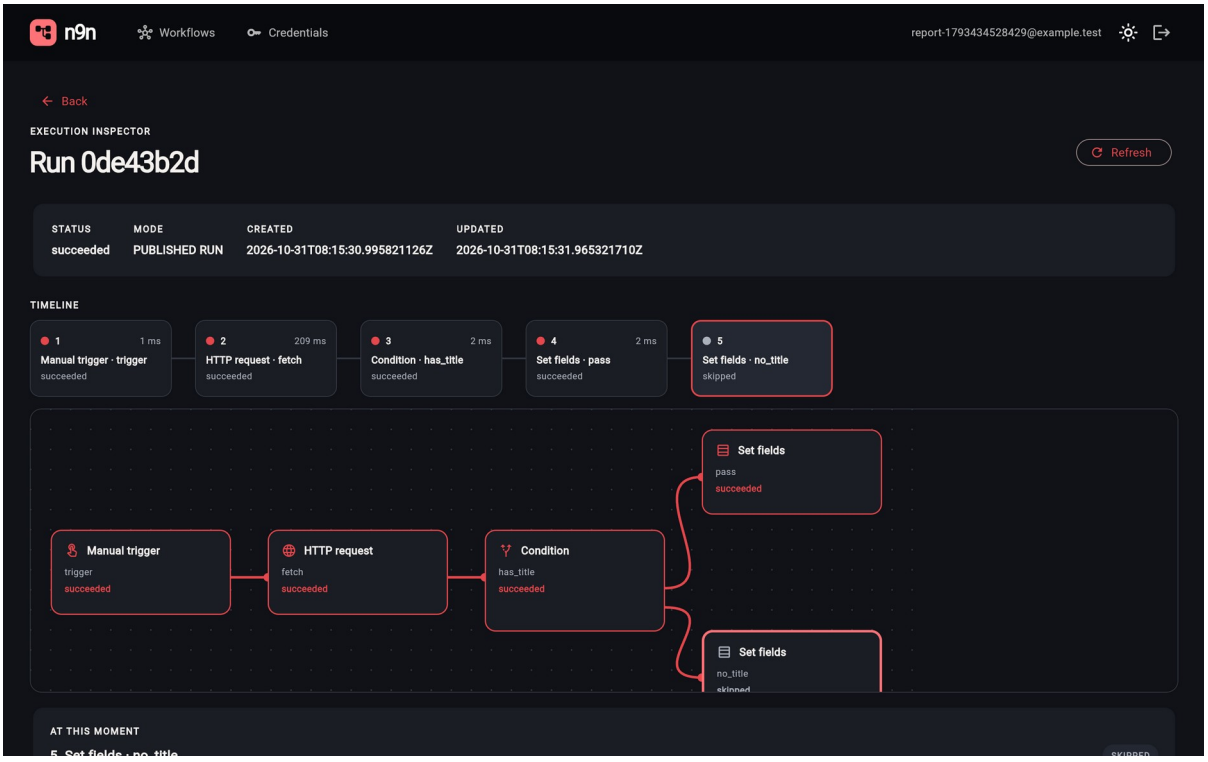


Fig. 4.6 Inspection of an SEO audit run of https://example.com/: the HTTP fetch succeeded and the page passed the title check

4.3 EXPERIMENTAL SETUP

All measurements in Sections 4.4 to 4.8 were taken on one machine (Apple M4, 10 cores, 16 GB, macOS) on 23 October 2026. PostgreSQL 16 ran in Docker; the Go API and the Rust worker (release build) ran as native processes against a database created for the experiments, so the figures include real network round trips between worker, API and database but not container overhead. The worker polled for jobs every 250 ms unless stated, and allowed private HTTP addresses so that a mock web site on the loopback address was reachable. A standard-library Python harness drove only the public API, the same surface the clients use; it is kept with the paper sources so the runs can be repeated.

4.4 END-TO-END LATENCY

The seven-node branching workflow of Fig. 4.7 (trigger, set-fields, two conditions and three set-fields leaves) was published and run 50 times sequentially with alternating inputs, so that both branches of each condition were taken. For each run the harness measured the time from issuing the run request to the first poll that observed a terminal status (poll interval 20 ms), and read back the recorded per-step durations (Table 4.2).

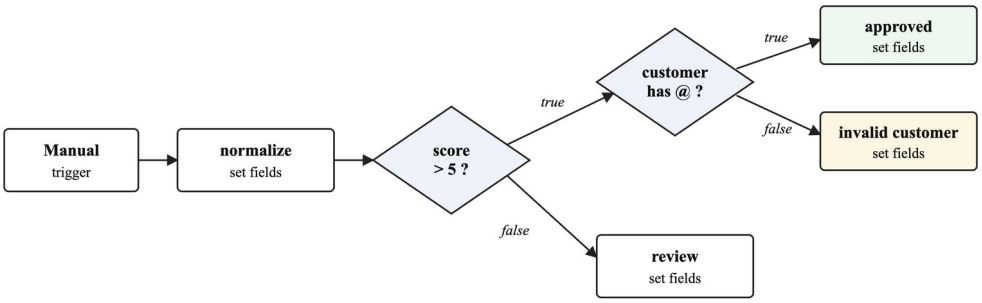


Fig. 4.7 Seven-node branching workflow used for the latency and throughput experiments

Table 4.2 End-to-end latency of the seven-node workflow (50 sequential runs)

Measure	Median	Mean	p95	Max
End-to-end wall time (ms)	259.6	260.6	290.2	295.6
Recorded step time, sum per run (ms)	7.0	6.6	11.0	16.0
Step: trigger (ms, n=50)	2.00	2.02	4.00	7.00
Step: normalize (ms, n=50)	1.50	1.60	3.00	7.00
Step: threshold (ms, n=50)	1.00	1.20	2.00	3.00
Step: known_customer (ms, n=25)	1.00	1.36	2.00	3.00
Step: approved (ms, n=17)	1.00	1.35	2.20	3.00

Measure	Median	Mean	p95	Max
Step: invalid_customer (ms, n=8)	1.00	1.00	2.00	2.00
Step: review (ms, n=25)	1.00	0.88	2.00	2.00

The recorded step time is small: the whole graph executes in a few milliseconds of worker time, so the end-to-end figure is set by the worker's poll interval, the 11 or 12 step reports (each an HTTP request from worker to API and a database insert) and the client's polling. Every run succeeded and recorded a status for each of the seven nodes, including the skipped leaves.

4.5 THROUGHPUT AND WORKER CONCURRENCY

To measure how the queue behaves under a burst, the worker was stopped, 100 runs of the same workflow were queued, and a worker was started with concurrency 1, 4 and 8 in turn. The makespan is taken from the database records: the interval from the earliest step start to the latest step finish across the 100 runs (Table 4.3, Fig. 4.8).

Table 4.3 Burst of 100 queued runs drained by one worker

Concurrency	Makespan (s)	Runs per second	Succeeded
1	1.40	71.3	100/100
4	0.56	178.5	100/100
8	0.63	158.6	100/100

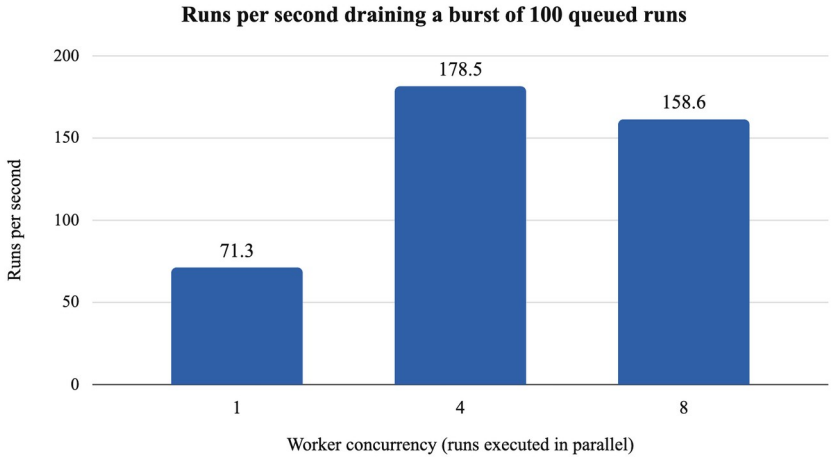


Fig. 4.8 Throughput by worker concurrency for a burst of 100 queued runs

Throughput rises from concurrency 1 to 4 and falls slightly at 8. This is consistent with the single API process and database being the shared resource: each run makes about 14 requests to the API (12 step reports plus a claim and a completion), so more worker tasks only add contention once the API is saturated. That no run is claimed twice is the property of the `SKIP LOCKED` claim and is covered by the API's concurrent-claim test.

4.6 REPLAY FROM A CHOSEN STEP

A nine-node chain (a trigger followed by eight set-fields nodes) was run 10 times; after each run the harness replayed it from its sixth node with an edited input. Each replay copied 5 upstream steps from the parent without executing them and executed 4, exactly the chosen node and its three descendants (Table 4.4, Fig. 4.9). Every replay recorded kind `replay` and its parent's identifier.

Table 4.4 Replay of a nine-node chain from its sixth node (10 repetitions)

Measure	Full run	Replay
Steps copied from parent	0	5
Steps executed	9	4
Wall time, median (ms)	281.6	228.8
Wall time, p95 (ms)	308.0	256.9

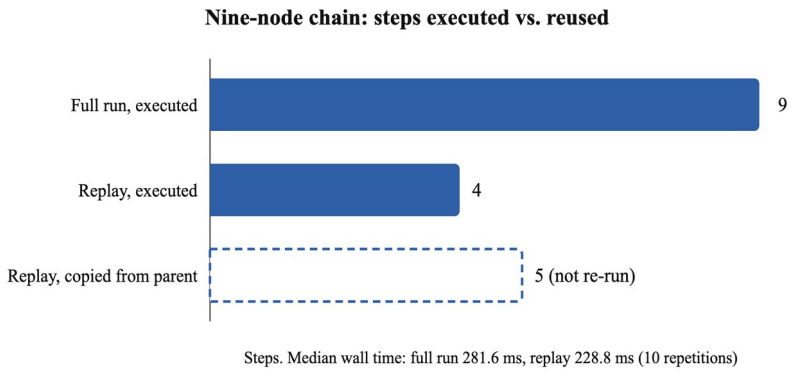


Fig. 4.9 Steps executed and reused when replaying the nine-node chain from its sixth node

4.7 TECHNICAL SEO AUDIT WORKFLOW

The audit workflow of Fig. 3.6, with a webhook trigger and a declared user agent, was activated and invoked once per target (Table 4.5). Three behaviours of the substrate shape the results. First, the HTTP node treats any non-2xx response as a failed step, so a 404 or a 301

ends the run as *failed* with the status in the attempt record. Second, redirects are not followed, so a moved page is reported as 301, which is correct for an audit. Third, the flaky target, which answers 500 twice and then 200, is reported as a pass with three recorded attempts, which separates an unstable server from a broken page without any workflow logic. The `robots.txt` and sitemap targets are reachable but are not HTML, so the title rule reports them as missing a title; a real audit needs a content-type check first. Among the public pages, three passed, one answered 301 to the request as sent (a finding about that site's canonical host), and one home page exceeded the HTTP node's 256 KiB response cap.

Table 4.5 Technical SEO audit, one run per target

Target	HTTP	Attempts	Fetch (ms)	Verdict
mock/ok	200	1	13	pass
mock/notitle	200	1	10	missing_title
mock/missing	404	1	2	fetch_failed
mock/moved	301	1	5	fetch_failed
mock/flaky (500, 500, 200)	200	3	765	pass
mock/robots.txt	200	1	6	missing_title
mock/sitemap.xml	200	1	5	missing_title
example.com	200	1	60	pass
iana.org	200	1	62	pass
python.org	200	1	118	pass
en.wikipedia.org/wiki/Main_Page	200	1	305	fetch_failed (over 256 KiB)
rust-lang.org	301	1	38	fetch_failed

4.8 FAULT INJECTION

The property that most separates n9n from the reviewed frameworks is what happens when a worker dies in the middle of an action. Three single-action workflows were run against a mock endpoint that answers after eight seconds. In each case the harness waited until the action's step was recorded as *running*, killed the worker with `SIGKILL`, and started a fresh worker half a second later; the fresh worker can claim the run only after the 60-second lease expires (Table 4.6).

Table 4.6 Recovery after the worker is killed mid-action (60 s lease)

Interrupted action	Final status	Seconds after kill	Step record
GET (idempotent by method)	succeeded	68.2	succeeded, attempt 2
POST, no idempotency key	uncertain	60.3	running, attempt 1; lease expired during a non-idempotent action
POST with idempotency key	succeeded	68.4	succeeded, attempt 2

A GET is idempotent, so the run was re-queued, the step was executed a second time and the run succeeded. A POST without an idempotency key is not safe to repeat, so the run was marked *uncertain* with the reason recorded and was never retried by the platform. A POST with an idempotency key was re-queued and succeeded like the GET. The uncertain verdict was recorded within about half a second of the lease expiring; the recoverable runs finished about eight seconds later, the time taken to execute the slow call again. Recovery time is therefore the lease length plus the action's own duration.

4.9 DEPLOYMENT VERIFICATION

The deployments were verified on 30 and 31 October 2026 with the end-to-end test suites (Table 4.7). The Docker Compose stack was also tested from a clean state with no `.env` file, as on a new machine: it generated its own keys, kept them across a restart and passed the end-to-end suite.

Table 4.7 Deployment verification

Deployment	Check	Result
Docker Compose (local)	All four services healthy; end-to-end API suite (7 groups)	Passed
Docker Compose (local)	Mobile bearer-session suite	Passed
Docker Compose, fresh machine	Keys generated, kept after restart, end-to-end suite	Passed
Docker Compose (local)	API reached after the API container moved to a new address	Passed
Cloudflare Worker (local, Wrangler)	End-to-end and mobile suites	Passed
Cloudflare Worker (production)	Health check and sign-in after secret repair	Passed

4.10 DISCUSSION

The measurements support three claims about the substrate. It is cheap: the worker spends single-digit milliseconds on a seven-node graph, and end-to-end latency is set by polling and reporting, both of which can be tuned. It is durable and replayable: every step, branch, skip and HTTP attempt is in the database, a run can be resumed from its records after a crash, and a finished run can be replayed from any node with reuse of upstream outputs. And it is honest about side effects: an interrupted non-idempotent action is surfaced as *uncertain* rather than silently repeated or silently dropped. These are the properties Chapter 2 found missing in the reviewed agent frameworks, and they are the properties an agent node will inherit by being an ordinary step.

The measurements also expose limits. Throughput is bounded by the single API process and the chatty step protocol; a deployment with several API replicas was not tested. The lease is fixed at 60 seconds, so recovery from a crash takes at least that long. The HTTP node's treatment of non-2xx responses as failures is right for integrations but means an audit workflow cannot branch on status inside the graph, and its 256 KiB response cap stopped the fetch of one large page; an option to pass non-2xx responses through as data, and a truncation mode for audits that need only the head of a page, are two small changes identified by these experiments.

4.11 LIMITATIONS

1. **Only the substrate is evaluated.** The builder, the explainer and the agent node are specified and feature-flagged, but no model was called. Nothing here measures the quality, cost or reliability of an agentic decision.
2. **Single machine, few trials.** Latency and replay figures come from 50 and 10 repetitions on one laptop; throughput and fault cases were run once per configuration, and no confidence intervals are reported.
3. **Synthetic workflows.** The graphs contain set-fields and condition nodes whose work is trivial, so the measurements characterise the engine, not realistic action latency.
4. **The SEO layer is a workflow, not a product.** The audit shows that technical checks can be expressed and recorded with the existing nodes; it does not crawl, score or assess content.
5. **Not checked.** Several API replicas, the Gmail trigger path under load, and behaviour during database failover were not exercised.

CHAPTER 5 – CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The reviewed literature gives workflow designers a rich set of patterns for making a model reason, call tools and cooperate with other models, and a consistent warning from every realistic benchmark that such agents remain unreliable. What it lacks is the systems layer beneath those patterns: a durable, replayable, side-effect-aware record of execution into which agentic decisions can be inserted without amplifying their failures.

n9n is built around that layer. Its deterministic substrate, implemented in Go and Rust on PostgreSQL with Flutter and React Native clients, validates workflows as acyclic single-trigger graphs, snapshots them immutably, executes them under leases with per-step and per-attempt records, replays them from any node, and classifies an interrupted non-idempotent action as uncertain rather than repeating it. On a local deployment it ran a seven-node workflow in a median of 260 ms end to end, drained 100 queued runs at up to 178.5 runs per second, reused 5 of 9 steps on replay, expressed a technical SEO audit with existing nodes, and recovered correctly from a worker killed mid-action in all three fault cases. The platform runs as one Docker Compose stack on any machine and publicly on Cloudflare Workers, with the Go and TypeScript APIs kept in sync by automated tests.

The contribution of this project is therefore the argument, the design and the measured substrate. The agentic claims remain to be earned by the experiments that the substrate now makes possible.

5.2 FUTURE SCOPE

The next step is to implement the agentic layer and evaluate it with metrics the substrate already records, so that each can be computed from the database without new instrumentation (Table 5.1).

Table 5.1 Evaluation plan for the agentic and SEO layers

Metric	Method
Builder validity rate	Fraction of generated graphs that pass the publish validator in round 1, 2 and 3, on a fixed set of requests covering every node type
Builder correctness	Fraction of validated graphs whose test run produces the expected result, judged by a person
Explainer agreement	On failed and uncertain runs with known causes, fraction where the stated cause matches; the enforced retry rule is never contradicted

Metric	Method
Reliability over repeats	pass^k for the builder and explainer over k = 5 identical requests
Cost and latency	Tokens and wall time per request from the usage log, against the per-user limit
Data policy	Redaction tests, and an audit that no outgoing request contains a known secret
Agent node under faults	Repeat the fault-injection experiment with an agent node calling a POST tool; expect uncertain and a preserved trace
SEO audit at scale	Run the audit over a sitemap of at least 500 URLs; precision and recall against a crawler baseline, attempts per URL, p95 time
Content assessment	Agreement between the agent node's relevance verdict and human raters, reported separately from the deterministic checks

Other planned work includes an approval node that pauses a run in the database until a person decides, a sandboxed WebAssembly code node, hosting the Rust worker next to the Cloudflare deployment, an option for the HTTP node to pass non-2xx responses through as data, and a truncation mode for large pages in SEO audits.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learning Representations (ICLR), 2023, arXiv:2210.03629.

[2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” 2023, arXiv:2302.04761.

[3] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), 2023, arXiv:2305.04091.

[4] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in Advances in Neural Information Processing Systems 36 (NeurIPS), 2023, arXiv:2305.10601.

[5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in Advances in Neural Information Processing Systems 36 (NeurIPS), 2023, arXiv:2303.11366.

[6] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, “Measuring and narrowing the compositionality gap in language models,” in Findings of the Assoc. Comput. Linguistics: EMNLP, 2023, arXiv:2210.03350.

[7] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in Proc. 41st Int. Conf. Machine Learning (ICML), 2024, arXiv:2402.01030.

[8] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” 2023, arXiv:2305.15334.

[9] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” 2023, arXiv:2308.08155.

[10] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “CAMEL: Communicative agents for ‘mind’ exploration of large language model society,” in Advances in Neural Information Processing Systems 36 (NeurIPS), 2023, arXiv:2303.17760.

[11] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” 2023, arXiv:2308.00352.

[12] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face,” 2023, arXiv:2303.17580.

[13] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” 2023, arXiv:2305.16291.

[14] n8n GmbH, “n8n: Workflow automation documentation.” [Online]. Available: https://docs.n8n.io/ (accessed Oct. 23, 2026).

[15] LangChain, Inc., “LangGraph documentation.” [Online]. Available: https://langchain-ai.github.io/langgraph/ (accessed Oct. 23, 2026).

[16] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in Proc. Int. Conf. Learning Representations (ICLR), 2024, arXiv:2307.13854.

[17] X. Liu et al., “AgentBench: Evaluating LLMs as agents,” in Proc. Int. Conf. Learning Representations (ICLR), 2024, arXiv:2308.03688.

[18] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, “GAIA: A benchmark for general AI assistants,” 2023, arXiv:2311.12983.

[19] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “τ-bench: A benchmark for tool-agent-user interaction in real-world domains,” 2024, arXiv:2406.12045.

[20] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” 2024, arXiv:2405.15793.

[21] P. Aggarwal, V. Murahari, T. Rajpurohit, A. Kalyan, K. Narasimhan, and A. Deshpande, “GEO: Generative engine optimization,” in Proc. 30th ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD), 2024, arXiv:2311.09735.

[22] L. Wang et al., “A survey on large language model based autonomous agents,” Frontiers of Computer Science, 2024, doi:10.1007/s11704-024-40231-1, arXiv:2308.11432.

[23] Z. Xi et al., “The rise and potential of large language model based agents: A survey,” 2023, arXiv:2309.07864.

[24] T. Masterman, S. Besen, M. Sawtell, and A. Chao, “The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey,” 2024, arXiv:2404.11584.

[25] P. Venkiteela, “n8n: An open-source workflow automation platform for enterprise integration and AI-driven orchestration,” 2025.

[26] D. R. Chirra, “How agentic AI is transforming workflow automation in 2025,” 2025.

[27] D. Pearson, S. Shapiro, E. S. Gonzalez Venegas, S. AlKhatib, and A. Pinzón Arzola, “Graph-based agentic AI with LangGraph: Workflow pathways for long-running stateful business processes,” 2026.

APPENDIX A – RUNNING THE PROJECT

The source code is available at https://github.com/JATIN-RAGHAV/n9n. The repository contains the API (`backend/`), the worker (`runner/`), the web editor (`web/`), the mobile app (`mobile/`), the Cloudflare deployment (`cloudflare/`), the API contract (`contracts/`), the tests (`tests/`) and the papers with their measurement harness (`paper/`).

A1 Local deployment

- Install Docker Desktop (about 10 GB of free disk space is needed for the first build).
- In the repository folder run `docker compose up --build -d --wait`.
- Open `http://localhost:8080` and create an account. No `.env` file is needed; the keys are generated on the first start.
- Stop with `docker compose down`; the data is kept in Docker volumes.

A2 Mobile app

- Install Expo Go on the phone and connect it to the same Wi-Fi network as the computer.
- In `mobile/` run `npm ci` and then `npm start`, and scan the QR code.
- In the app, set Settings, Server address to `http://<computer LAN address>:8080`, or to `https://n9n.walrus.codes` for the public deployment.

A3 Tests

- `make test` runs the Go, Rust and Flutter suites; `python3 tests/e2e_smoke.py` runs the end-to-end API suite against a running stack.
- The measurements of Chapter 4 can be reproduced with `paper/research-paper-src/experiments/run_experiments.py`, as described in `paper/README.md`.

APPENDIX B – PAPERS PRODUCED

Two papers were written as part of this project by Anmol Bhardwaj, Avi Gupta, Jatin Raghav and Keshav Garg (Department of Computer Science and Engineering (AI/ML), MIET Meerut):

1. “AI Workflow Automation using Agentic Orchestration: A Review,” a review of the agentic AI and workflow automation literature and the research gaps for hybrid architectures, under the supervision of Mr. Ankit Aggarwal and Ms. Mohini Singh.
2. “AI Workflow using Agentic Orchestration and SEO,” an IEEE conference-format paper (11 pages) that extends the review to twenty-four works, presents the n9n design and reports the measured evaluation of the implemented substrate summarised in Chapter 4.